

Table of Contents

ИНСТРУКЦИЯ ПО ВНЕДРЕНИЮ

ТЗ-5 — LLM Contract & DM-Agent Repair (v0.1)

Проект: ENIGMA v0.5.3.1.9 **Документ:** Инструкция по ремонту LLM-контура и DM-агента **Тип:** Техническое задание на миграцию **Цель:** восстановить контракт «LLM = голос, не судья», закрыть NameError/ImportError, транслировать forbidden-список в промпт

Принцип (из Causal Authority Map): - LLM = voice layer (рендерит computed state, не создаёт behavioral truth) - Python = causal core (состояние рождается из причинности) - No silent failures (warnings вместо except: pass) - No fallback reality (MockProvider исключён из production)

0. Контекст

0.1. Диагноз

LLM-контур ENIGMA страдает от **3 разрывов контракта**:

Разрыв	Симптом	Примеры
Prompt ↔ Parser mismatch	LLM не знает формат, парсер не восстанавливает	dm_response поле, forbidden-список
Silent validation failure	Валидатор глотает ошибки	recent_text=None, _contains_dialog ложное срабатывание
Dead code contamination	Орфан-модули создают ложное впечатление системы	ResponseParser, NPCResponseValidator, VerbalStance.to_prompt_line

0.2. 17 дефектов, сгруппированных в 3 Patch Set

Patch Set	Дефектов	Принцип	Время
A — Pure Fix (NameError, ImportError, contract)	7	Восстановить заявленное поведение	2 часа

Patch Set	Дефектов	Принцип	Время
B — Authority Enforcement (prompt ↔ parser sync)	5	LLM знает формат, валидатор ловит	3 часа
C — Design Change / Cleanup	5	ADR-level или dead code removal	2 часа

1. Порядок применения

STEP 1 → dm_agent.py:136 (A1 — несуществующий импорт)
STEP 2 → scene_outcome_builder.py:397 (A2 — NameError npc_id)
STEP 3 → dm_agent.py:674-689 (A3 — JSON recovery)
STEP 4 → dm_agent.py:692, 723 (A4 — recent_text проброс)
STEP 5 → response_validator.py:129 (A5 — _contains_dialog типе)
STEP 6 → response_validator.py:99 (A6 — _contains_non_russian ASCII)
STEP 7 → dm_agent.py:670,709,771,793 (A7 — max_tokens → settings)
STEP 8 → dm_system.txt (B1 — формат ответа в промпт)
STEP 9 → dm_system.txt (B2 — forbidden-список в промпт)
STEP 10 → dm_contract_builder.py (B3 — forbidden-блок динамически)
STEP 11 → mock_provider.py + factory.py (B4 — Mock exclusion в production)
STEP 12 → response_validator.py (B5 — _force_static убрать)
STEP 13 → validation tests

Design Change / Cleanup (C): - C1: Удалить world_sim_agent.py (мёртвый) - C2: Удалить parser.py (мёртвый) - C3: Удалить npc_response_validator.py (мёртвый) - C4: VerbalStance.to_prompt_line — подключить или удалить - C5: Удалить мёртвые LLM-провайдеры (vllm, openai, anthropic, ollama)

2. PATCH SET A — Pure Fix (7 шагов, ~2 часа)

STEP 1 — backend/app/agents/dm_agent.py:134-139 (A1)

ЗАМЕНИ:

```

    if _scene_state:
        try:
            from app.services.scene.scene_state_manager import
SceneStateManager
            _scene_block =
SceneStateManager.get_scene_description(_scene_state) + "\n\n"
        except Exception:
            pass

```

НА:

```
# A1-FIX: правильный путь импорта (без '.scene.' в середине).  
# Убран try/except: pass — он маскировал ModuleNotFoundError.  
if _scene_state:  
    from app.services.scene_state_manager import SceneStateManager  
    _scene_block = SceneStateManager.get_scene_description(_scene_state)  
+ "\n\n"
```

STEP 2 — backend/app/services/verbalization/scene_outcome_builder.py:397 (A2)

ЗАМЕНИ:

```
line = f"- {npc.name or npc_id} {intent_desc}"
```

НА:

```
# A2-FIX: npc_id не определён в scope → npc.npc_id.  
line = f"- {npc.name or npc.npc_id} {intent_desc}"
```

STEP 3 — backend/app/agents/dm_agent.py:674-689 (A3)

Назначение: Recovery JSON-ответа LLM, если нет поля dm_response.

ЗАМЕНИ блок извлечения dm_text:

```
if isinstance(raw, str):  
    try:  
        result = json.loads(raw)  
        if not isinstance(result, dict):  
            result = {"dm_response": str(result).strip()}  
    except Exception:  
        result = {"dm_response": raw.strip()}  
else:  
    result = raw if isinstance(raw, dict) else {"dm_response": str(raw)}  
  
dm_text = result.get("dm_response", "")
```

НА:

```
# A3-FIX: recovery из альтернативных JSON-схем.  
# Раньше: только dm_response → пусто, если LLM дал NPC-схему.  
if isinstance(raw, str):  
    try:  
        result = json.loads(raw)  
    except Exception:  
        result = None  
        dm_text = raw.strip()  
elif isinstance(raw, dict):
```

```

    result = raw
else:
    result = None
    dm_text = str(raw).strip()

if result is not None:
    if not isinstance(result, dict):
        dm_text = str(result).strip()
    elif "dm_response" in result:
        dm_text = result["dm_response"].strip()
    elif "speech" in result or "action" in result:
        # Legacy NPC-схема — восстанавливаем
        logger.warning(f"[DM_AGENT] LLM returned NPC-schema, recovering:
{list(result.keys())}")
        dm_text = (result.get("speech") or result.get("action") or "").strip()
    else:
        # Неизвестная схема — ищем любое длинное строковое значение
        dm_text = ""
        for k, v in result.items():
            if isinstance(v, str) and len(v) > 20:
                logger.warning(f"[DM_AGENT] unknown schema, using {k}:
{result}")
                dm_text = v.strip()
                break
else:
    pass # dm_text уже установлен выше

```

STEP 4 — backend/app/agents/dm_agent.py:692, 723 (A4)

Назначение: Пробросить recent_text в ResponseValidator.validate.

ЗАМЕНИ (строка 692):

```
validation = validator.validate(dm_text)
```

НА:

```

# A4-FIX: передаём recent text для проверки повторов.
_recent = self._get_last_dm_response()
validation = validator.validate(dm_text, recent_text=_recent)

```

Аналогично для строки 723.

ДОБАВИТЬ метод _get_last_dm_response:

```

def _get_last_dm_response(self) -> Optional[str]:
    """Возвращает последний ДМ-ответ из истории."""
    try:
        # Из player_avatar dialog_journal
        if hasattr(self, '_memory_manager') and self._memory_manager:

```

```

        journal = self._memory_manager.get_recent_dm_responses(limit=1)
        if journal:
            return journal[0]
    except Exception as e:
        logger.warning(f"[DM_AGENT] could not get recent DM response: {e}")
    return None

```

STEP 5 — backend/app/services/verbalization/response_validator.py:129 (A5)

ЗАМЕНИ:

```

def _contains_dialog(self, text: str) -> bool:
    return '""' in text or '«' in text or '—' in text[:10]

```

НА:

```

def _contains_dialog(self, text: str) -> bool:
    """A5-FIX: убрано ложное срабатывание на тире (DM-нарратив)."""
    # Прямая речь — только кавычки. Тире в начале — DM-стиль, не диалог.
    return '""' in text or '«' in text

```

STEP 6 — backend/app/services/verbalization/response_validator.py:99-112 (A6)

ЗАМЕНИ:

```

def _contains_non_russian(self, text: str) -> bool:
    """Китайские иероглифы или доминирующий не-русский текст."""
    if self._CJK_PATTERN.search(text):
        return True
    alpha_total = sum(1 for c in text if c.isalpha())
    if alpha_total > 10:
        russian_like = sum(
            1 for c in text
            if c.isalpha() and (c.isascii() or '\u0400' <= c <= '\u04ff')
        )
        if russian_like / alpha_total < 0.6:
            return True
    return False

```

НА:

```

def _contains_non_russian(self, text: str) -> bool:
    """A6-FIX: ASCII считается НЕ russian-like. Моск-ответы отклоняются."""
    if self._CJK_PATTERN.search(text):
        return True
    alpha_total = sum(1 for c in text if c.isalpha())
    if alpha_total <= 10:

```

```

    return False
    russian_chars = sum(1 for c in text if '\u0400' <= c <= '\u04ff')
    ascii_chars = sum(1 for c in text if c.isascii() and c.isalpha())
    # Если русских < 60% — подозрительно
    if russian_chars / alpha_total < 0.6:
        return True
    # Если ASCII > 30% — вероятно английский (вкл. Mock-маркеры)
    if ascii_chars / alpha_total > 0.3:
        return True
    # Явная защита от Mock-утечек
    if "[Mock]" in text or "[mock]" in text:
        return True
    return False

```

STEP 7 — backend/app/agents/dm_agent.py:670, 709, 771, 793 (A7)

ЗАМЕНИ (4 места):

```
params=GenerationParams(max_tokens=220),
```

НА:

```

    # A7-FIX: max_tokens из settings, не хардкод.
    params=GenerationParams(max_tokens=settings.dm_max_tokens),

```

В backend/app/core/config.py добавить:

```

class Settings(BaseSettings):
    ...
    dm_max_tokens: int = 220 # было хардкожено в 4 местах

```

3. PATCH SET B — Authority Enforcement (5 шагов, ~3 часа)

STEP 8 — backend/prompts/dm_system.txt (B1)

Назначение: Специфицировать формат ответа LLM.

ДОБАВИТЬ в конец файла:

ФОРМАТ ОТВЕТА

Отвечай ТОЛЬКО валидным JSON:

```

{
  "dm_response": "твой рассказ 2-3 предложения",
  "npc_reactions": [{"npc_id": "maid_lusya", "speech": "реплика в кавычках"}]
}

```

Если NPC не отвечает — npc_reactions пустой массив.
НЕ оборачивай в markdown-блоки. НЕ добавляй текст до или после JSON.

STEP 9 — backend/prompts/dm_system.txt (B2)

Назначение: Forbidden-список в промпт (preventive, не только post-hoc).

ДОБАВИТЬ перед блоком «ФОРМАТ ОТВЕТА»:

КАТЕГОРИЧЕСКИЕ ЗАПРЕТЫ

1. НЕ придумывай события, которых не было в вводных данных.
2. НЕ меняй намерение NPC (intent) — оно задано в блоке «Ключевые NPC».
3. НЕ описывай мысли NPC — только внешне наблюдаемые проявления.
4. НЕ повторяй сказанное в предыдущих ходах.
5. НЕ задавай вопросы игроку — ты рассказчик, не собеседник.

Нарушение → ответ отклоняется, игрок видит «Ничего не произошло.»

STEP 10 — backend/app/services/verbalization/dm_contract_builder.py (B3)

Назначение: Динамически генерировать forbidden-блок из контракта.

НАЙТИ метод build в DMContractBuilder, **ДОБАВИТЬ** перед сборкой:

```
# B3-FIX: forbidden-блок из контракта (single source of truth).
if self._forbidden_tuple:
    parts.append("### КАТЕГОРИЧЕСКИЕ ЗАПРЕТЫ:")
    for i, forbidden in enumerate(self._forbidden_tuple, 1):
        parts.append(f"{i}. НЕ {forbidden}.")
    parts.append("")
    parts.append("Нарушение → ответ отклоняется.")
    parts.append("")
```

STEP 11 — backend/app/services/llm/factory.py:93-94 + mock_provider.py (B4)

Назначение: MockProvider исключён из production (No fallback reality).

ЗАМЕНИ в factory.py:93-94:

```
case ProviderType.МОСК:
    return create_mock_provider()
```

НА:

```
case ProviderType.MOCK:
    # B4-FIX: MockProvider excluded from production (No fallback reality).
    from app.core.config import settings
    if settings.environment == "production":
        raise RuntimeError(
            "[LLM_FACTORY] CRITICAL: MockProvider in production. "
            "Mock is simulation of simulation — no place in runtime reality. "
            "Set settings.environment='development' to allow."
        )
    return create_mock_provider()
```

В backend/app/core/config.py добавить:

```
class Settings(BaseSettings):
    ...
    environment: str = "production" # production / development / test
```

STEP 12 — backend/app/services/verbalization/response_validator.py:155-171 (B5)

Назначение: Убрать `_force_static` — он ломает Инвариант 2 (заменяет реальные перемещения NPC).

ЗАМЕНИ:

```
def _force_static(self, text: str) -> str:
    replacements = {
        "подходит": "смотрит на",
        "отходит": "отворачивается",
        "идёт": "стоит",
        ...
    }
    for old, new in replacements.items():
        text = text.replace(old, new)
    return text
```

НА:

```
def _force_static(self, text: str) -> str:
    """B5-FIX: DEPRECATED. Убраны replacements — они ломают Инвариант 2.
```

*Раньше: заменял 'подходит' → 'смотрит на', если can_move=False.
Проблема: DM-контракт содержит РЕАЛЬНЫЕ перемещения NPC (через SceneChange).
Замена маскировала реальное движение.*

Теперь: по-ор. Если NPC не может двигаться, SceneChange не создаётся,

и DM-контракт не содержит информации о движении.
"""

return text

4. PATCH SET C — Cleanup (5 пунктов, ~2 часа)

C1 — Удалить backend/app/agents/world_sim_agent.py

Проверка:

```
grep -rn "WorldSimulationAgent" backend/ --include="*.py" | grep -v __pycache__ |  
grep -v test
```

Если только определение + упоминание в game_loop_builder.py:70 —
удалить файл + убрать импорт.

C2 — Удалить backend/app/services/llm/parser.py

Проверка:

```
grep -rn "ResponseParser" backend/ --include="*.py" | grep -v __pycache__
```

Если только определение — удалить.

C3 — Удалить backend/app/services/verbalization/npc_response_validator.py

Проверка:

```
grep -rn "NPCResponseValidator" backend/ --include="*.py" | grep -v __pycache__
```

Если только определение — удалить.

C4 — verbal_stance.py — подключить или удалить

Проверка:

```
grep -rn "to_prompt_line\|VerbalStance" backend/ --include="*.py" | grep -v  
__pycache__ | grep -v test
```

Вариант А (подключить): в scene_outcome_builder.to_dm_prompt_block
добавить:

```
stance_line = npc.stance.to_prompt_line() if npc.stance else ""  
if stance_line:  
    line += f" [{stance_line}]"
```

Вариант В (удалить): убрать VerbalStance класс + вычисление stance в
scene_outcome_builder.py:606-613.

Рекомендация: Вариант А (stance — ценная информация для LLM).

C5 — Удалить мёртвые LLM-провайдеры

Если cloud-провайдеры не планируются — удалить: -

backend/app/services/llm/vllm_provider.py -

backend/app/services/llm/openai_provider.py -

backend/app/services/llm/anthropic_provider.py -

backend/app/services/llm/ollama_provider.py

Обновить `__init__.py` — убрать импорты.

Если планируются — оставить, но добавить ADR «Cloud providers roadmap».

5. STEP 13 — backend/tests/test_tz5_llm_contract.py (NEW)

"""Integration tests for T3-5 LLM Contract Repair."""

import pytest

from pathlib **import** Path

class TestPatchA_NameErrors:

def test_no_wrong_import_in_dm_agent(self):

 path = Path(__file__).resolve().parents[2] / "app" / "agents" / "dm_agent.py"

 source = path.read_text(encoding="utf-8")

assert "from app.services.scene.scene_state_manager" **not in** source, \

 "Wrong import path in dm_agent (A1 not applied)"

def test_no_npc_id_undefined(self):

 path = Path(__file__).resolve().parents[2] / "app" / "services" / "verbalization" /

"scene_outcome_builder.py"

 source = path.read_text(encoding="utf-8")

Не должно быть "or npc_id}" без npc.npc_id

assert "or npc_id}" **not in** source, \

 "NameError npc_id in scene_outcome_builder (A2 not applied)"

class TestPatchA_ValidatorFixes:

def test_contains_dialog_no_dash(self):

from app.services.verbalization.response_validator **import**

ResponseValidator

Нужно создать contract для теста — пропускаем если сложно

Проверяем source code

 path = Path(__file__).resolve().parents[2] / "app" / "services" / "verbalization" /

"response_validator.py"

 source = path.read_text(encoding="utf-8")

assert "'—" in text[:10]" **not in** source, \

 "_contains_dialog still checks dash (A5 not applied)"

def test_non_russian_rejects_mock(self):

```
    path = Path(__file__).resolve().parents[2] / "app" / "services" / "verbalization" /  
    "response_validator.py"  
    source = path.read_text(encoding="utf-8")  
    assert "[Mock]" in source, \  
        "_contains_non_russian doesn't check [Mock] (A6 not applied)"
```

```
class TestPatchB_PromptSync:
```

```
    def test_dm_system_has_format(self):  
        path = Path(__file__).resolve().parents[2] / "prompts" / "dm_system.txt"  
        source = path.read_text(encoding="utf-8")  
        assert "ФОРМАТ ОТВЕТА" in source, "Format spec missing in prompt (B1  
not applied)"  
        assert "dm_response" in source, "dm_response field not documented (B1 not  
applied)"
```

```
    def test_dm_system_has_forbidden(self):  
        path = Path(__file__).resolve().parents[2] / "prompts" / "dm_system.txt"  
        source = path.read_text(encoding="utf-8")  
        assert "КАТЕГОРИЧЕСКИЕ ЗАПЕТЫ" in source, "Forbidden list missing  
(B2 not applied)"
```

```
class TestPatchB_MockExclusion:
```

```
    def test_factory_rejects_mock_in_production(self):  
        path = Path(__file__).resolve().parents[2] / "app" / "services" / "llm" /  
        "factory.py"  
        source = path.read_text(encoding="utf-8")  
        assert "production" in source and "MockProvider" in source, \  
            "MockProvider not blocked in production (B4 not applied)"
```

```
class TestPatchC_DeadCode:
```

```
    def test_world_sim_agent_removed(self):  
        path = Path(__file__).resolve().parents[2] / "app" / "agents" /  
        "world_sim_agent.py"  
        assert not path.exists(), "world_sim_agent.py should be removed (C1)"
```

```
    def test_parser_removed(self):  
        path = Path(__file__).resolve().parents[2] / "app" / "services" / "llm" /  
        "parser.py"  
        assert not path.exists(), "parser.py should be removed (C2)"
```

```
    def test_npc_response_validator_removed(self):  
        path = Path(__file__).resolve().parents[2] / "app" / "services" / "verbalization" /  
        "npc_response_validator.py"  
        assert not path.exists(), "npc_response_validator.py should be removed (C3)"
```

6. СВОДКА

Patch	Step	Тип	Время
A1	dm_agent.py: 136 import	Pure Fix	5 мин
A2	scene_outcome_builder.py:397	Pure Fix	2 мин
A3	dm_agent.py: 674-689 JSON recovery	Pure Fix	30 мин
A4	dm_agent.py: 692 recent_text	Pure Fix	20 мин
A5	response_validator.py:129 dash	Pure Fix	5 мин
A6	response_validator.py:99 ASCII	Pure Fix	15 мин
A7	dm_agent.py max_tokens × 4	Pure Fix	15 мин
B1	dm_system.txt format	Authority	15 мин
B2	dm_system.txt forbidden	Authority	15 мин
B3	dm_contract_builder.py forbidden block	Authority	30 мин
B4	factory.py + mock_provider exclusion	Authority	30 мин
B5	response_validator.py _force_static	Authority	15 мин
C1-C5	dead code removal	Cleanup	1 час
Test	test_tz5_llm_contract.py	—	1 час

Итого Patch Set A+B+C: ~7 часов.

7. ЧТО ПАТЧ ГАРАНТИРУЕТ

✓ Вводная сцена — непустая (правильный импорт) ✓ NPC без name — не крашит DMFrame ✓ JSON без dm_response — восстанавливается ✓ recent_text — передаётся, повторы ловятся ✓ DM-нарратив с тире — не считается диалогом ✓ Mock-ответы — отклоняются валидатором ✓ max_tokens — из settings ✓ LLM знает формат ответа (JSON с dm_response) ✓ LLM знает forbidden-список (preventive) ✓ MockProvider — недоступен в production ✓ Мёртвые модули удалены

КОНЕЦ ИНСТРУКЦИИ ТЗ-5